

PROJECT ID:



University of Manouba
National School of Computer Science



SUMMER INTERNSHIP REPORT

Advanced AI Orchestration

Automated Conversational Evaluation Pipeline

AUTHOR

Omar Brika



COMPANY	Focus Corporation
RESPONSIBLE'S NAME	Focus Corporation R&D
PROFESSIONAL SUPERVISOR	Ayman Mkaouar
ADDRESS	Chotrana, Ariana, Tunisia

Academic Year 2025 / 2026

Appréciations et signature de l'encadrant

DATE

SIGNATURE AND STAMP

Résumé

Valider les réponses d'un assistant vocal embarqué à grande échelle, dans de nombreuses langues et sous des contraintes de latence proches de celles d'un environnement automobile, est un problème coûteux à résoudre par des méthodes manuelles ou par des scripts de correspondance de mots-clés. Ce stage, réalisé au sein du département R&D de Focus Corporation, propose un banc de test conversationnel automatisé dans lequel un unique modèle de langage léger, exécuté localement via llama.cpp, joue tour à tour les rôles de Prompteur, de Routeur et de Juge face à un Dispositif Sous Test (DUT), le tout orchestré par un graphe LangGraph. Neuf modèles candidats (familles Qwen3.5, gemma-4, Phi et Ministral) ont été évalués sur un jeu de 216 cas de test multilingues couvrant 14 domaines automobiles, avec une instrumentation fine de la latence par type d'appel. Les résultats désignent gemma-4-E2B-it comme le meilleur compromis pour la production (95,8 % de précision, 0,734 s), Qwen3.5-4B restant préférable lorsque la précision maximale prime. Le stage inclut enfin une évaluation de sécurité autorisée du matériel Jetson ciblé, ayant révélé deux vulnérabilités critiques d'OpenSSH.

Mots-clés : orchestration de modèles de langage, LangGraph, agents conversationnels, benchmark multilingue, sécurité embarquée

Abstract

Validating an in-vehicle voice assistant’s responses at scale, across many languages, and under automotive-grade latency constraints is expensive to solve with manual QA or keyword-matching scripts. This internship, carried out in Focus Corporation’s R&D department, builds an automated conversational evaluation pipeline in which a single lightweight LLM, served locally through llama.cpp, successively plays the roles of Prompter, Router, and Judge against a Device Under Test (DUT), the whole orchestrated as a LangGraph state graph. Nine candidate models (the Qwen3.5, gemma-4, Phi, and Ministral families) were benchmarked on 216 multilingual test cases spanning 14 automotive domains, with latency instrumented per call type. Results identify gemma-4-E2B-it as the best production trade-off (95.8% accuracy, 0.734 s average latency), with Qwen3.5-4B preferred when maximum accuracy is required. The internship also includes an authorized security assessment of the target Jetson-class hardware, which uncovered two critical/high-severity OpenSSH vulnerabilities.

Keywords: LLM orchestration, LangGraph, conversational agents, multilingual benchmark, embedded security

ملخص

يُعد التحقق من صحة استجابات المساعد الصوتي داخل المركبة على نطاق واسع، وبعده لغات، وفي ظل قيود زمنية تقترب من بيئة السيارة الفعلية، مشكلة مكلفة عند الاعتماد على الاختبار اليدوي أو نصوص المطابقة بالكلمات المفتاحية. يقدم هذا التقرير، المنجز ضمن قسم البحث والتطوير في شركة Focus Corporation، خط أنابيب تقييم حوار آلي يقوم فيه نموذج لغوي واحد خفيف الوزن، يُشغل محليًا عبر llama.cpp، بأداء أدوار المُحفِّز والموجِّه والحَكَم على التوالي أمام الجهاز الخاضع للاختبار، ضمن مخطط تنفيذ من نوع LangGraph. جرى تقييم تسعة نماذج مرشحة (عائلات Qwen3.5 و gemma-4 و Phi و Ministral) على 216 حالة اختبار متعددة اللغات تغطي 14 مجالاً من مجالات السيارة، مع قياس دقيق لزمان الاستجابة حسب نوع كل نداء. تُظهر النتائج أن نموذج gemma-4-E2B-it يقدم أفضل توازن للإنتاج (دقة 95.8% وزمن استجابة 0.734 ثانية)، بينما يُفضّل Qwen3.5-4B عند الحاجة إلى أقصى دقة ممكنة. كما يتضمن التقرير تقييمًا أمنيًا مُصرِّحًا به لعتاد Jetson المستهدف، كشف عن ثغرتين خطيرتين في OpenSSH.

الكلمات المفتاحية: تنسيق النماذج اللغوية الكبيرة، LangGraph، الوكلاء الحواريون، معيار متعدد اللغات، الأمن المضمّن

Acknowledgements

I would like to sincerely thank Mr. Ayman Mkaouar, my professional supervisor at Focus Corporation, for his continuous guidance, his fast and detailed feedback on every iteration of the evaluation pipeline, and for trusting me with a project that combined applied AI engineering with a genuine security assessment. I am equally grateful to the Focus Corporation R&D team for their welcome and technical support throughout this internship, and to my academic supervisor at the National School of Computer Science (ENSI) for their guidance on this report. Finally, I thank my family and friends for their constant encouragement during this internship.

Contents

Résumé	ii
Abstract	iii
ملخص	iv
Acknowledgements	v
Contents	vi
List of Figures	viii
List of Tables	ix
List of Acronyms	x
Introduction	1
1 General Context	2
1.1 Internship Framework	2
1.2 About the Company	2
1.3 Problematic	2
1.4 Study of the Existing	3
1.5 Mission	3
1.6 Proposed Solution Diagram	4
2 State of the Art: Technologies	5
2.1 LLMs and Small Language Models.	5

2.2	llama.cpp and llama-server	5
2.3	LangChain and LangGraph	6
2.4	Pydantic / JSON Constrained Output	6
2.5	LangChain Callback Instrumentation	7
2.6	Edge Hardware (NVIDIA Jetson-class, CUDA 12.6)	7
2.7	OpenSSH Security Fundamentals.	7
3	Project Implementation	9
3.1	Graph Design.	9
3.2	Test-Suite Engineering.	10
3.3	Latency Instrumentation	11
3.4	Automation Tooling	12
3.5	The Benchmark Campaign	13
	Conclusion et perspectives	18
	Bibliography	20
A	Security Assessment of the Target Edge Hardware	21

List of Figures

1.1	Proposed solution: the Prompter/DUT/Judge closed loop (the four-stage flow described above).....	4
3.1	Overall model performance: accuracy against turn-match rate, per candidate model.....	14
3.2	Latency breakdown by call type (initial prompter / routing / judgment).....	14
3.3	Per-domain accuracy across candidate models.....	15
3.4	Average per-call latency by model.....	15

List of Tables

3.1	Test-suite composition (<code>Json_samples/samples.json</code>).	11
3.2	Benchmark results across the nine candidate models.	13
3.3	Model-selection scoring: the two composite-score versions.	16
A.1	Security assessment findings on the in-scope Jetson-class host.	22

List of Acronyms

ASR	Automatic Speech Recognition
BFCL	Berkeley Function-Calling Leaderboard
CUDA	Compute Unified Device Architecture
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DoS	Denial of Service
DUT	Device Under Test
ENSI	National School of Computer Science (École Nationale des Sciences de l'Informatique)
FP16	16-bit Floating-Point Precision
GGUF	GPT-Generated Unified Format — llama.cpp's model container format
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IFEval	Instruction-Following Evaluation (benchmark)
JSON	JavaScript Object Notation
LLM	Large Language Model
LPDDR5	Low-Power Double Data Rate 5 (memory)
MMMLU	Multilingual Massive Multitask Language Understanding (benchmark)
NSCA	Nagios Service Check Acceptor
PoC	Proof of Concept
Q4_K_M	4-bit K-quantization, medium variant (llama.cpp)
QA	Quality Assurance
R&D	Research and Development
RCE	Remote Code Execution
SSH	Secure Shell
TOPS	Tera-Operations Per Second
TTS	Text-to-Speech
UMA	University of Manouba
VAD	Voice Activity Detection
VLAN	Virtual Local Area Network
VRAM	Video Random-Access Memory

Introduction

Automotive voice assistants are evaluated against hundreds of natural-language commands, in dozens of languages, before they can be shipped inside a vehicle. Historically, this evaluation has relied on human testers reading scripted scenarios aloud, or on brittle keyword-matching scripts that cannot tell a correct paraphrase from a wrong action. Neither approach scales to the combination of domains, languages, and conversational turns that a modern in-cabin assistant must support, and neither can run inside the tight latency budget of the embedded hardware the assistant will eventually ship on.

This report describes the work carried out during a first-year internship in the R&D department of Focus Corporation, whose objective was to replace that manual loop with an automated conversational evaluation pipeline. A single lightweight large language model (LLM), served locally through llama.cpp and orchestrated as a LangGraph state graph, plays three cognitive roles in sequence – Prompter, Router, and Judge – against a Device Under Test (DUT), closing the test loop without a human in it. Nine small-to-medium open-weight models were benchmarked on a purpose-built, 216-case multilingual automotive test suite, and the resulting latency and accuracy data were used to recommend a model for deployment on Focus Corporation’s target Jetson-class edge board. As a secondary, security-oriented deliverable, an authorized vulnerability assessment of that same target board was carried out, following a standard reconnaissance-enumeration-exploitation methodology.

The remainder of this report is organized as follows. Chapter 1 sets the general context of the internship and the problem it addresses. Chapter 2 reviews the state of the art of the technologies the pipeline is built on. Chapter 3 details the implementation – the orchestration graph, the test suite, the latency instrumentation, the automation tooling, and the benchmark campaign and its results. The report then closes with a discussion of what was achieved and what remains as future work, followed by Appendix A, which documents the security assessment as a self-contained deliverable.

CHAPTER 1

General Context

1.1 Internship Framework

This report covers a first-year internship completed within the curriculum of the National School of Computer Science (ENSI), University of Manouba, for the 2025/2026 academic year. The internship was hosted by the Research & Development department of Focus Corporation, based in Chotrana, Ariana, Tunisia, under the professional supervision of Mr. Ayman Mkaouar. Its scope was defined jointly by the department's need for a scalable QA process for an automotive voice assistant, and the academic requirement to apply software-engineering and AI methods to a real, open-ended problem over the course of the internship.

1.2 About the Company

Focus Corporation's R&D department works on applied artificial-intelligence systems for the automotive industry, with a focus on in-vehicle voice interaction. The department evaluates and integrates conversational AI components – from natural-language understanding to text-to-speech – before they are embedded in a vehicle's infotainment stack. The project described in this report fits into that pipeline at the quality-assurance stage: before any voice-assistant build is signed off, its behaviour across the vehicle's supported domains and languages must be checked, and that is precisely the gap this internship's pipeline fills.

1.3 Problematic

The central problem is one of scale under constraint. A production automotive voice assistant must be checked against a large and growing matrix of user intents (navigation, media, climate control, phone, and more), a double-digit list of supported languages, and conversations that can run to several turns when the assistant needs to ask for clarification. Every new build of the assistant must be re-validated against that matrix,

and the validation itself has to run fast enough to fit into a development cycle, and eventually needs to reflect the latency budget of the embedded, GPU-shared hardware the assistant will ship on. The question this internship set out to answer is: how can this coverage be reproduced automatically, cheaply, and consistently across languages, without needing a human tester for every domain $\times$ language $\times$ build combination?

1.4 Study of the Existing

Three existing approaches were considered and found insufficient on their own. Traditional, human-led manual QA – a human tester speaks or types each command and manually judges the assistant’s reply – remains the ground-truth method in terms of judgment quality, but it does not scale: it is slow, expensive to repeat on every build, and its throughput cannot grow with the number of supported languages and domains. Scripted keyword-matching, where a test harness checks the assistant’s transcript for expected keywords or intents, scales well and runs instantly, but it is brittle: a semantically correct paraphrase that omits the expected keyword is marked as a failure, and, more importantly, keyword matching cannot distinguish a wrong action (the assistant did something plausible but incorrect) from a hallucination (the assistant fabricated a response unrelated to the request) – both simply register as “no match.” Cloud LLM-based judging, sending the assistant’s transcript to a large hosted model for a PASS/FAIL judgment, solves the semantic-matching problem, but introduces a network round-trip and a per-call cost that make it unsuitable both for a fast, everyday grading loop and, more fundamentally, for representing the latency behaviour of the final embedded, offline deployment.

None of the three approaches doubles as a driver of the conversation itself – they all assume a human or another scripted harness is already producing the next voice command. The pipeline described in this report was designed to remove that assumption as well.

1.5 Mission

The mission set for this internship was to design and build a single, self-contained evaluation pipeline in which one lightweight LLM, running entirely on local, OpenAI-compatible inference (llama.cpp), plays all three roles a human tester or a keyword script would otherwise require: generating the next natural-language voice command (Prompter), deciding whether the Device Under Test’s (DUT) reply concludes the interaction or calls for a follow-up (Router), and issuing a semantically grounded PASS/FAIL verdict with a written justification (Judge). A second objective, added once the pipeline itself worked, was to benchmark a representative pool of small open-weight models against a purpose-built multilingual automotive test suite, and to translate

the resulting accuracy/latency data into a concrete model recommendation for Focus Corporation’s target edge hardware.

1.6 Proposed Solution Diagram

Figure 1.1 shows the closed conversational loop between the Prompter and the Judge around the Device Under Test, in four stages. Initiation: the Prompter uses the test case’s configuration (intent, target language) to generate the first natural voice command sent to the DUT. Evaluation: the Prompter ingests the DUT’s response and the accumulated conversation history. Routing: based on that analysis, the Prompter’s own decision engine determines whether the conversation requires further clarification or has reached a natural conclusion. Handoff: once the conversation is judged complete, the full state – the entire transcript – is forwarded to the Judge, which issues the final PASS/FAIL verdict.

NOTE ON THE DIAGRAM

The diagram already reflects an implementation choice made during the build: Routing is not a separate box because it is not a separate graph node in the shipped code — it is the second kind of LLM call the Prompter itself makes. This design simplification, and how it differs from the three-node architecture originally planned, is discussed in full in §3.1 (Graph Design).

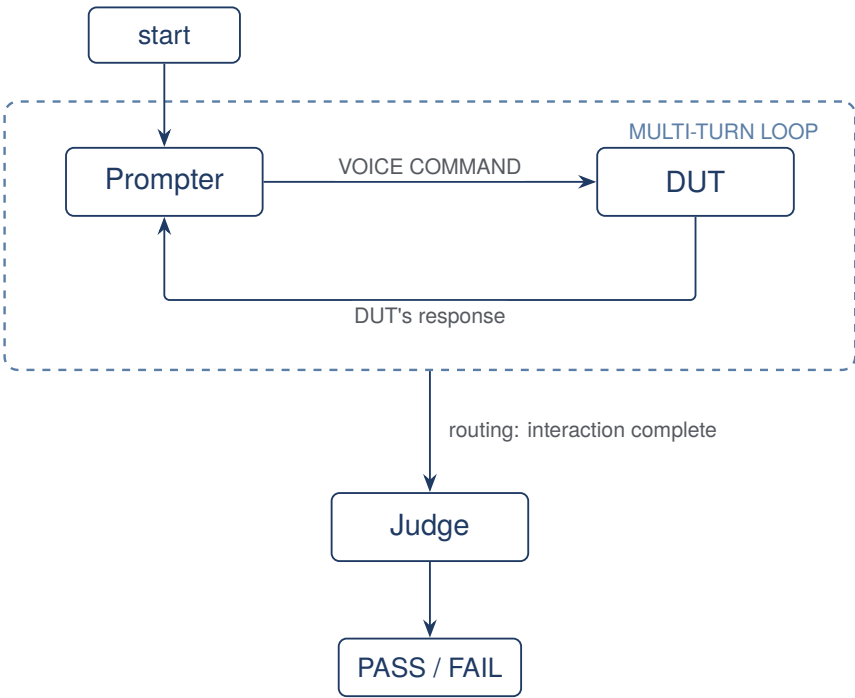


Figure 1.1. Proposed solution: the Prompter/DUT/Judge closed loop (the four-stage flow described above).

CHAPTER 2

State of the Art: Technologies

2.1 LLMs and Small Language Models

Large language models are increasingly used not only to generate free text but to act as function-calling, instruction-following controllers inside a larger system – exactly the role the Prompter, Router, and Judge need to fill here. Three families of public benchmarks were used to shortlist candidate models before the internship’s own benchmark campaign began: IFEval, which checks whether a model can respect a rigid output contract (essential, since every Prompter and Judge call must return parseable JSON); MMMLU and INCLUDE, which measure whether a model’s reasoning quality survives translation into the languages the test suite covers; and τ^2 -Bench / BFCL, which measure a model’s ability to act as an agent inside a changing conversational state – the closest public proxy for the Router’s job. Published scores gathered during candidate shortlisting (e.g. Qwen3.5-4B: IFEval 89.8, MMMLU 76.1, τ^2 -Bench 79.9; Qwen3.5-2B: IFEval 78.6, MMMLU 63.1, τ^2 -Bench 48.8) informed the initial candidate pool before empirical, task-specific numbers replaced them as the deciding factor in Chapter 3.

Running one of these models at full precision is not an option on a shared, power- and memory-constrained edge board. All nine candidate models are therefore served in Q4_K_M – a 4-bit, per-block quantization scheme (llama.cpp’s GGUF format) that keeps the most sensitive tensors at a slightly higher effective precision than the bulk of the weights. This roughly quarters the model’s memory footprint relative to FP16 at a comparatively small accuracy cost, which is what makes it realistic to keep one of these models resident, or to swap between them, inside the RAM/VRAM budget shared with the vehicle’s other AI workloads.

2.2 llama.cpp and llama-server

llama.cpp is a C/C++ inference engine for GGUF-quantized models that runs efficiently on CPU and consumer/embedded GPUs alike, including the CUDA 12.6 stack quoted for the target Jetson-class board. Its `llama-server` binary exposes an OpenAI-compatible

`/v1/chat/completions` HTTP endpoint, which is the detail that makes the whole architecture possible: the pipeline’s LangChain `ChatOpenAI` client can talk to a fully local model over `http://localhost:<port>/v1` exactly as it would to a cloud provider, so nothing in `graph.py` or `nodes.py` needs to know or care that the “API” it is calling is actually a local process. `run.sh` launches `llama-server` with a 1024-token context and full GPU layer offload, waits for the server’s `/health` endpoint to report `ok` before starting the benchmark, and tears the server down again once the Python harness exits – which is also what lets the benchmark queue move from one candidate model to the next unattended.

2.3 LangChain and LangGraph

LangChain supplies the thin client layer (`ChatOpenAI`) and the callback interface the pipeline uses to talk to `llama-server` and to instrument every call. LangGraph, built on top of it, supplies the actual orchestration: a `StateGraph` is a graph of named node functions that each receive and return a partial update to a single shared, typed state object, wired together by edges that can be conditional on that state. This is the right abstraction for a conversational evaluator because the control flow is not linear – the same Prompter logic must be revisited every time the DUT asks for clarification, and the graph only exits the loop once a routing flag says the conversation is over. `graph.py` builds exactly that: two nodes (`prompter`, `judge`), wired by conditional edges that read the state’s `routing_flag` and route back into `prompter` on `AWAITING_DUT`, forward into `judge` on `PROCEED_TO_JUDGE`, or straight to `END` on `TERMINATED`. LangGraph’s `recursion_limit` (10 for the offline benchmark graph) is what protects a run from an infinite prompter-to-prompter loop if the routing logic never sets one of the terminal flags.

2.4 Pydantic / JSON Constrained Output

Both the Prompter and the Judge are asked, in their prompts, to return a single JSON object rather than free text, and `parsers.py` parses that text into `PrompterSchema`/`JudgeSchema` Pydantic models before the graph does anything with it – `JudgeSchema.verdict`, for instance, is typed as a two-value literal, so a validated instance is guaranteed to be either `PASS` or `FAIL` and nothing else. This is not the fully constrained decoding described as an aspiration in the original design notes (grammar-constrained generation that would make invalid JSON literally unreachable); the pipeline instead re-parses the model’s raw text output defensively. `extract_json_from_text()` in `parsers.py` tries three fallback strategies in turn – a fenced ````json` block, a bare `{...}` object, and a trailing-comma cleanup pass – before giving up and falling back to a naive text heuristic. That fallback chain is a pragmatic response to Q4_K_M-quantized small models, which do not always respect a JSON-only instruction as reliably as a full-precision frontier model would.

2.5 LangChain Callback Instrumentation

LangChain exposes a `BaseCallbackHandler` interface with hooks fired at defined points in an LLM call's lifecycle (`on_chat_model_start`, `on_llm_start`, `on_llm_end`, `on_llm_error`). `LLMTimeTracker` (`llm_tracker.py`) subclasses this handler to record a timestamp when a call starts and compute its exact duration when it ends, storing each call as a `duration/tokens/tags/run_id` record in a flat list – the `tags` field being the hook that lets the rest of the pipeline tell which of the three logical call types (`initial_prompter`, `routing_voice_cmd`, `judgment`) a given measurement belongs to, since every call to `llm.invoke()` in `nodes.py` is issued with a matching `tags` argument. Implementing this correctly required matching the exact keyword-only signatures LangChain's callback mixins expect – a point noted explicitly in the project's own engineering log after an initial implementation broke on a positional-argument mismatch. The tracker itself never touches durations beyond timing and tagging; the aggregation into the three named averages (`avg_initial_prompter_time`, `avg_routing_voice_cmd_time`, `avg_judgment_call_time`) is left to `reporter.py`.

2.6 Edge Hardware (NVIDIA Jetson-class, CUDA 12.6)

The pipeline's model-selection outcome is meant to run on the Jetson-class board specified by Focus Corporation's R&D team: an NVIDIA Ampere-architecture GPU (2048 CUDA cores, 64 Tensor Cores) rated at up to 275 TOPS of AI performance, a 12-core Arm CPU at 2.2 GHz, 64 GB of shared LPDDR5 RAM/VRAM, and CUDA 12.6. The specification explicitly notes that this compute is not dedicated to the voice-assistant pipeline alone but shared with the vehicle's other AI workloads – which is precisely why model size and per-call latency, not just accuracy, were tracked as first-class benchmark metrics throughout Chapter 3, rather than being an afterthought applied only at the end.

2.7 OpenSSH Security Fundamentals

SSH (Secure Shell) is the remote-administration protocol most embedded Linux boards, Jetson-class or otherwise, expose for field maintenance and over-the-air management, which makes its server implementation (OpenSSH's `sshd`) part of the attack surface of any device that ships with it enabled. Two classes of vulnerability are relevant to the assessment in Appendix A (page 21): memory-safety issues triggered by signal handling, where a signal interrupts execution at an unsafe point and re-enters non-reentrant code, corrupting memory in a way that can be turned into remote code execution; and resource-exhaustion issues, where a protocol handshake step allocates memory that a malicious client can force the server to keep re-allocating without ever completing the handshake, exhausting the host's memory. Both classes are unauthenticated – no valid credentials

are needed to trigger either path – which is what elevates their severity for a device likely to be reachable on a vehicle’s internal network.

CHAPTER 3

Project Implementation

3.1 Graph Design

The evaluation state is a single typed dictionary, `BrainState` (`state.py`), threaded through the whole graph: the session’s `intent` and `target_language`, the running `recent_messages` transcript, the pre-scripted `dut_script` / `dut_script_idx` pair that stands in for the Device Under Test in the offline benchmark, a `routing_flag` restricted to the three values `AWAITING_DUT` / `PROCEED_TO_JUDGE` / `TERMINATED`, the eventual `final_verdict` / `final_reasoning`, and bookkeeping fields (`turn_count`, `expected_turns`, `llm_logs`) used purely for the benchmark’s own metrics.

Rather than the three separate node types described in the original architecture brief, the shipped offline graph (`graph.py`) is built with exactly two `StateGraph` nodes, `prompter` and `judge`. A conditional edge reads `routing_flag` after every `prompter` call and sends execution back into `prompter` on `AWAITING_DUT`, forward to `judge` on `PROCEED_TO_JUDGE`, or straight to `END` on `TERMINATED`; `judge` always edges to `END`. The Router’s decision – is this interaction over, or does it need another turn? – is not a separate node call; it is folded into the `Prompter` node’s own second LLM call (`create_prompter_node` in `nodes.py`), tagged `routing_voice_cmd`, which is prompted with the conversation history and the DUT’s last reply and asked to return both the next command and an `is_interaction_complete` flag in the same JSON object. This is a deliberate simplification relative to the “Agentic Memory & Multi-Turn Routing” milestone in the original project design notes, which specified a standalone Router node as a distinct logic gate; folding it into the `Prompter`’s second call instead halves the number of LLM round-trips per non-terminal turn, at the cost of a router that cannot later be independently swapped for a smaller, faster model, as the original two-tier design intended (see the Conclusion). The `system_design.svg` architecture diagram itself already reflects this simplification: it draws only a `Prompter` box and a `Judge` box, with the routing decision explicitly described in its own caption as something the “`Prompter`’s decision engine” performs, not a third box.

Two further implementation details are worth recording precisely because they only surface on close reading of the code, not from the diagram or the benchmark

output. First, the canonical `conversation-eval-pipeline/src/schemas.py` currently ships a `PrompterSchema` with a single field, `text_command`; `nodes.py`, however, still reads `parsed.is_interaction_complete` off that same object on the multi-turn branch of `prompter_node`. The two files are mutually consistent in the working copy of the pipeline that actually produced the benchmark results discussed below – that copy’s `schemas.py` also declares an `is_interaction_complete` boolean field – but that field is missing from the copy of `schemas.py` currently checked into the `conversation-eval-pipeline` tree that this report was asked to treat as the primary source. It is a small but real regression between the version that was actually run and the version presently committed, and a concrete illustration of why every number in this report was re-derived from the raw JSON benchmark output rather than trusted from any single source file. Second, the offline graph’s recursion guard is a bare `recursion_limit` of 10 passed to `graph.invoke()` with no exception handling around it; the live variant, discussed next, handles the equivalent failure mode explicitly instead of relying on the limit never being hit.

A second, `live/` variant of the graph (`live_graph.py`, `live_nodes.py`, `live_state.py`) exists for driving an actual, non-scripted DUT rather than a pre-written `dut_script`. It keeps the Prompter/Judge split but inserts two additional nodes between them and the outside world: `voice_cmd`, which turns the Prompter’s text command into audio via an injected voice-command function, and `asr`, which turns the DUT’s audio reply back into text via an injected speech-recognition function – both are dependency-injected callables rather than hard-coded services, so the same graph can be pointed at a real text-to-speech/ASR stack or at a mock pair for local testing. `LiveConversationGraph` also compiles its graph with an in-memory checkpointer, keyed by a per-conversation thread id, which is what lets it recover the last good state on failure rather than losing the run outright: its `invoke()` sets `recursion_limit` to three times the maximum allowed turns and explicitly catches a `GraphRecursionError`, in which case it reads back the last checkpointed turn count and returns a `FAIL` verdict with an explanatory reasoning string, instead of letting the exception propagate. The live Prompter node also sidesteps the `is_interaction_complete` inconsistency described above entirely: it infers completion from whether the model returned an empty `text_command`, rather than reading a separate boolean field – a cleaner contract that the offline benchmark harness would benefit from adopting.

3.2 Test-Suite Engineering

The test suite (`Json_samples/samples.json`) holds 216 cases, each a `{name, intent, language, domain, dut_script, expected_verdict, expected_turns}` record. `dut_script` is a pre-written list of one or two DUT replies: this is what lets the offline benchmark run unattended and reproducibly against nine different models – the DUT’s behaviour is fixed by the test case, so only the Prompter/Router/Judge triangle under test can vary between runs.

Table 3.1. Test-suite composition (`Json_samples/samples.json`).

Property	Count	Share
Total test cases	216	100.0 %
Expected PASS	192	88.9 %
Expected FAIL	24	11.1 %
Multi-turn (2-turn) cases	24	11.1 %
Languages	12	—
Domains	14	—

The 216 cases are spread evenly across 12 languages (18 cases each: ar-SA, de-DE, en-US, es-ES, fr-FR, it-IT, ja-JP, ko-KR, pt-BR, ru-RU, tr-TR, zh-CN) and 14 automotive domains – CLIMATE, MEDIA, NAVIGATION and PHONE at 24 cases each, and AMBIENT_LIGHTING, DIGITAL_ASSISTANT, DRIVING_ASSISTANCE, HVAC, MAINTENANCE, RADIO, SEAT_COMFORT, VEHICLE_CONTROL, VEHICLE_STATUS and WEATHER at 12 cases each. All 24 two-turn cases expect a PASS verdict, specifically exercising the Router’s clarification-loop logic; the 24 FAIL cases are all single-turn.

The 24 FAIL cases were engineered around a distinction raised explicitly during the internship’s engineering review: a wrong answer is not necessarily a hallucination, but every hallucination is a wrong answer, so the two need separate test cases rather than a single generic “fail” bucket. Twelve MEDIA cases are tagged **WrongAction**: the DUT performs a plausible but incorrect action for the request (e.g. the intent “Play jazz” answered with “Opening the sunroof.”) – a failure of action selection, not of comprehension. Twelve CLIMATE cases are tagged **Hallucination**: the DUT fabricates an unrelated, non-sequitur response to a well-formed command (e.g. the intent “Turn on the heater” answered with “I have added a space heater to your Amazon cart.”) – a failure to stay grounded in the conversation at all. Distinguishing the two matters for the Judge’s reasoning field, and, downstream, for deciding whether a model’s failures are a prompt-tuning problem (wrong action) or a more fundamental groundedness problem (hallucination).

The CLIMATE domain’s PASS cases were also deliberately written with implicit rather than explicit intent – “I am cold” rather than “Turn on the heater” – specifically to test whether a model can infer the correct vehicle action from a driver’s natural phrasing instead of a literal command. §3.5 shows this single design choice is what separates the benchmarked models most sharply.

3.3 Latency Instrumentation

Every `llm.invoke()` call in `nodes.py` is wrapped by a small tracking helper that reads the `LLMTimeTracker`’s call count before and after the call and sums the durations of

whatever new entries appeared – robust to a call silently retrying internally, since every retry would still show up as its own timed entry. Each node call is issued with a tag identifying which of three metric buckets it belongs to: `initial_prompter` for the very first Prompter call in a conversation (no history yet), `routing_voice_cmd` for every subsequent Prompter call (deciding completion and generating the next command in the same call), and `judgment` for the single terminal Judge call. `runner.py` reads these per-call tags back out of each test case’s `llm_logs` to compute, per test case, `initial_prompter_time`, `routing_voice_cmd_time` and `judgment_call_time` (plus matching call counts), which `reporter.py` then sums and averages across the whole 216-case suite into the report’s headline `avg_initial_prompter_time`, `avg_routing_voice_cmd_time` and `avg_judgment_call_time` figures – the three numbers behind the latency-breakdown charts in §3.5.

3.4 Automation Tooling

`installModels.sh` reads `models.conf` – a flat `category|repo_id|filename` list, with a quant placeholder expanded to the requested quantization (Q4_K_M by default) – and pulls each GGUF file from Hugging Face, skipping files that already exist and retrying only the ones that failed, into a `model-dir/{reasoning, non_reasoning}/` split. That split matters at run time: the Qwen3.5 and gemma-4 families are hybrid reasoning models and are launched with their reasoning trace explicitly disabled for this task, while the Phi and Ministral families are launched unmodified. `run.sh` then iterates every model file in both subdirectories, starts `llama-server` for it (1024-token context, full GPU layer offload), polls its `/health` endpoint (up to 30 retries at 2 s, i.e. up to 60 s of load time – deliberately generous, since some of the larger candidate models take noticeably longer than a tighter timeout would allow to finish loading), runs the Python benchmark harness to completion, and tears the server down again before moving to the next model – the loop that produced all nine `benchmark_report_*.json` files from a single, unattended invocation. A signal trap additionally ensures that both the Python harness and the `llama-server` process are killed if the whole run is interrupted, rather than leaking a server process on the port the next iteration needs.

Most of this tooling exists because of a specific, dated request from the professional supervisor rather than a design choice made in isolation. The June 29 sync asked, among other things, to add the Gemma 4 family to the testing queue, to track the three latency metrics separately (already discussed in §3.3), to grow the evaluation dataset to at least 200 samples, and to build a Bash script capable of iterating the model queue with the Python harness supporting the necessary command-line arguments. Each of those four requests maps directly onto, respectively, the gemma-4-E2B-it / gemma-4-E4B-it rows in the results table below, the `initial_prompter` / `routing_voice_cmd` / `judgment` tags described in §3.3, the 216-case `samples.json` (grown from an original 50-case pilot suite), and the `run.sh` / `model-name` / `port` / `test-samples` / `verbose` argument pairing described above.

Table 3.2. Benchmark results across the nine candidate models (216 test cases, Q4_K_M, temperature 0.0). The two models carried forward to the deployment recommendation are shaded.

Model	Accuracy	Avg. latency (s)	Turn-match
Minstral-3-3B-Instruct	36.11 %	1.589	98.61 %
Minstral-3-8B-Instruct	60.65 %	2.976	100.00 %
Phi-3.5-mini-instruct	86.11 %	2.043	100.00 %
Phi-4-mini-instruct	89.81 %	0.737	100.00 %
Qwen3.5-2B	90.28 %	0.643	100.00 %
Qwen3.5-4B	96.30 %	1.156	100.00 %
Qwen3.5-9B	93.98 %	1.568	100.00 %
gemma-4-E2B-it	95.83 %	0.734	100.00 %
gemma-4-E4B-it	95.83 %	1.164	100.00 %

3.5 The Benchmark Campaign

All nine models were benchmarked under identical conditions: `llama-server` on localhost, 1024-token context, Q4_K_M quantization, temperature 0.0 (deterministic decoding, so repeated runs are directly comparable), one model resident at a time, run against the full 216-case suite. Table 3.2 reports the resulting accuracy, average per-test latency, and turn-match rate for each model, and Figure 3.1 plots accuracy against turn-match visually.

Accuracy ranges from 36.1% (Minstral-3-3B) to 96.3% (Qwen3.5-4B) – a 60-point spread across nine models at broadly similar parameter counts (2B–9B), which shows how much instruction-following quality varies within the “small model” category independently of raw size: the 9B Qwen3.5-9B (94.0%) is outperformed by its own 4B sibling (96.3%), and the 8B Minstral (60.6%) trails every Qwen3.5 and gemma-4 variant regardless of size. Turn-match – whether the model produced exactly the number of conversational turns the test case expected – is 100% for every model except Minstral-3-3B (98.6%), showing that the routing logic itself (deciding when to stop) is largely solved even for the weaker models; the accuracy gap is concentrated in the Judge’s semantic verdict and the Prompter’s action selection, not in turn-taking.

Latency is dominated by the Judge call for every model: it accounts for between 47.8% (gemma-4-E2B-it) and 63.7% (Qwen3.5-4B) of a test case’s total LLM time, computed directly from each model’s reported judgment and total call times, because the Judge is the only call given the full conversation history and asked to produce free-form reasoning before its verdict, rather than a short, templated command. Figures 3.2 and 3.4 show this breakdown per model; the two Minstral models stand out with a disproportionate routing-call cost, consistent with the accuracy problems these models show throughout the suite.

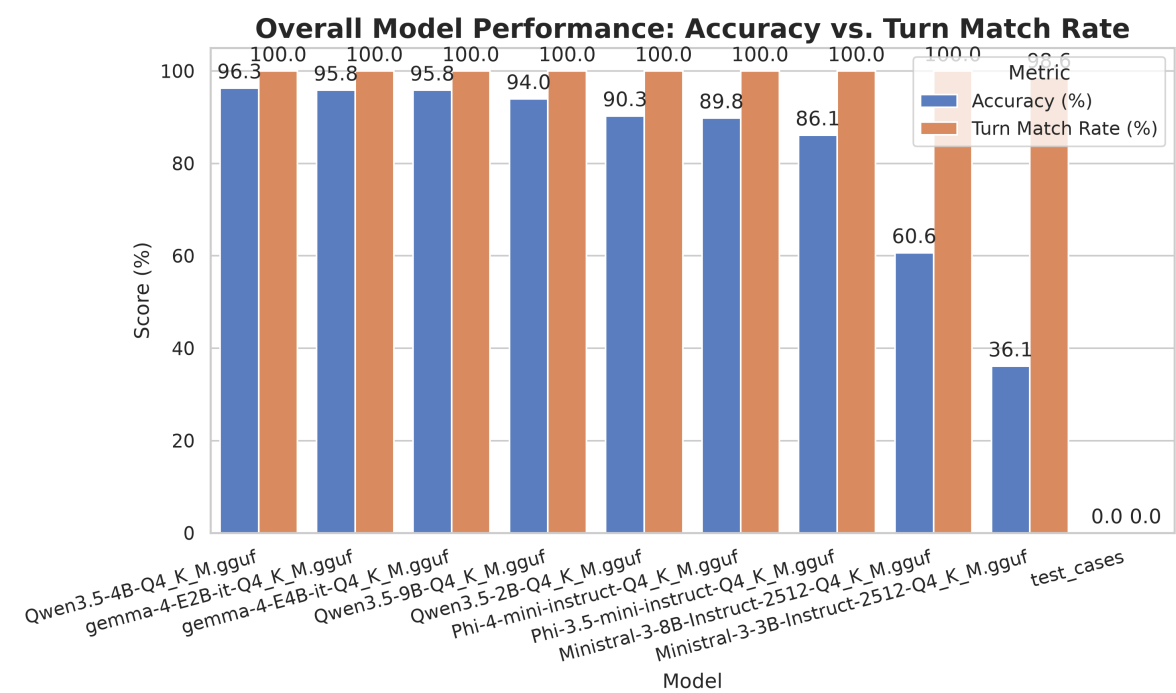


Figure 3.1. Overall model performance: accuracy against turn-match rate, per candidate model.

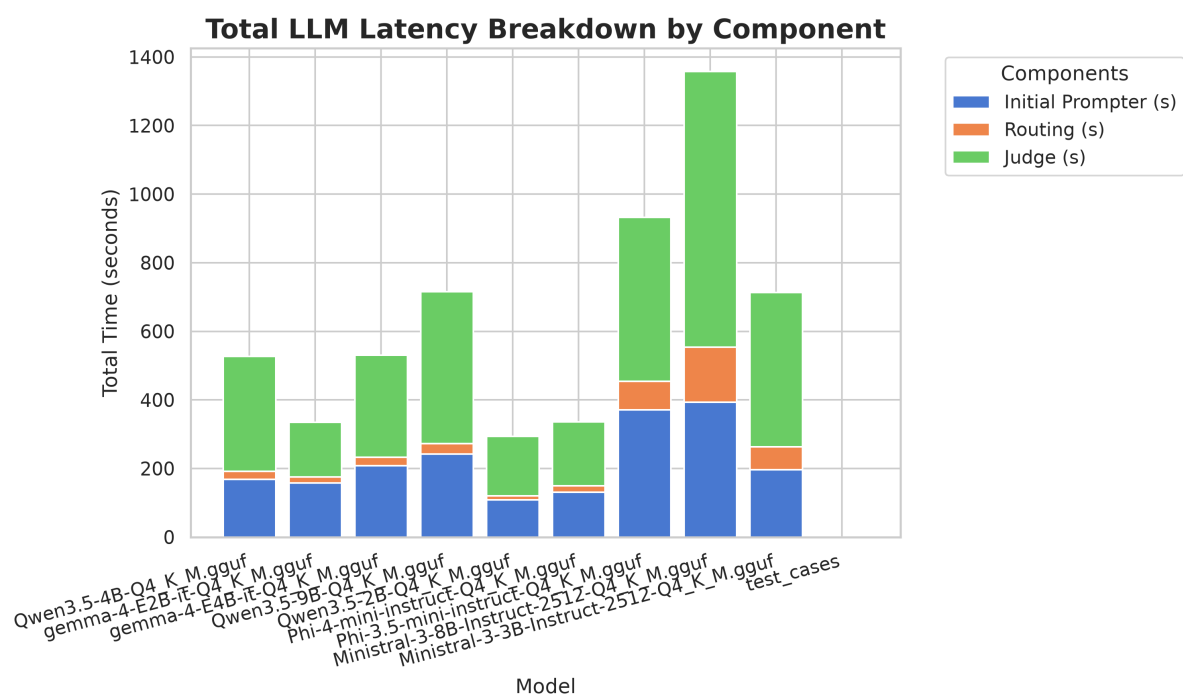


Figure 3.2. Latency breakdown by call type (initial prompter / routing / judgment).

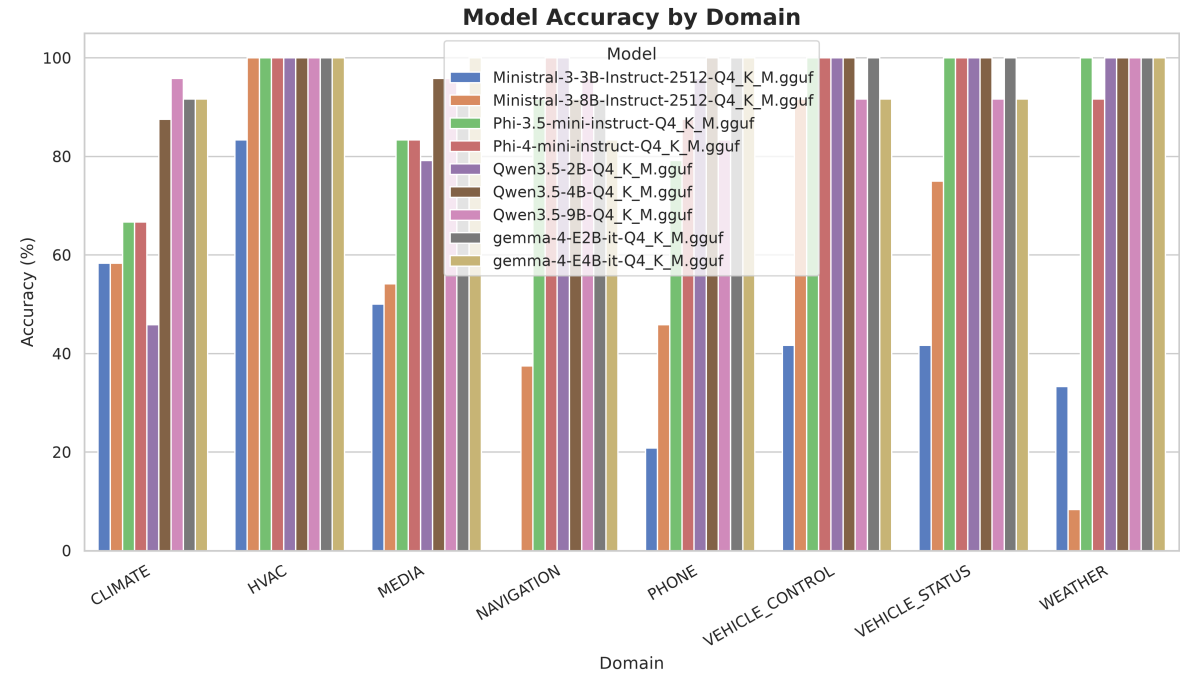


Figure 3.3. Per-domain accuracy across candidate models.

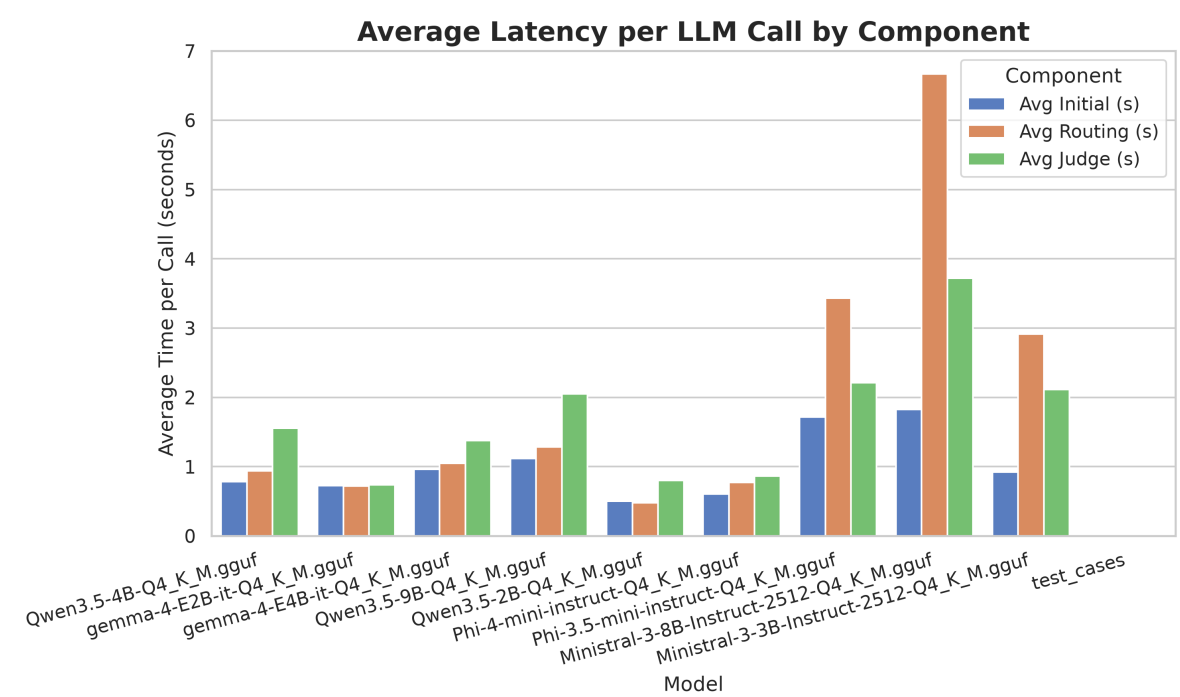


Figure 3.4. Average per-call latency by model.

Table 3.3. Model-selection scoring: the two versions of the composite `jetson_score`, which differ only in whether the Judge call’s latency is counted.

Version	Latency scope	Rank 1	Latency	Score
A	Production only (no Judge cost)	Qwen3.5-2B	0.980 s	0.960
B	Full end-to-end (incl. Judge)	gemma-4-E2B-it	0.734 s	0.973

Domain-level accuracy (Figure 3.3) is where the benchmark discriminates most clearly between the viable and non-viable candidates. Safety- and comfort-adjacent domains that admit only one plausible correct action per intent – HVAC, DRIVING_ASSISTANCE, AMBIENT_LIGHTING, SEAT_COMFORT – sit at or near 100% for every model from Phi-3.5-mini upward; only the two Ministral models fall meaningfully short there (e.g. DRIVING_ASSISTANCE: 8.3% for Ministral-3-3B against 100% for six of the other seven models). CLIMATE is the clearest differentiator among the otherwise-strong candidates, specifically because its PASS cases use implicit rather than explicit intent phrasing (§3.2): Qwen3.5-2B drops to 45.8% on CLIMATE alone despite being the second-fastest model overall, while Qwen3.5-4B (87.5%), gemma-4-E2B-it and gemma-4-E4B-it (91.7% each) and Qwen3.5-9B (95.8%) handle the same cases comfortably – meaning a deployment that leans on Qwen3.5-2B for its speed advantage would need either additional prompt engineering around implicit climate commands or a dedicated intent-classification pre-step to close that specific gap. MAINTENANCE is a second, milder weak spot across the whole board (peaking at 91.7% for the strongest models, and as low as 41.7–50% for the Phi family), suggesting the domain’s phrasing or vocabulary is generally harder for this model class regardless of family.

Two scoring passes were run over this same data to turn it into a deployment recommendation, differing only in whether the Judge call’s latency is counted. Both use the same composite formula:

```

norm_speed      = 1 - (latency - min_latency) / (max_latency - min_latency)
norm_accuracy   = (accuracy - min_accuracy) / (max_accuracy - min_accuracy)
jetson_score    = 0.6 × norm_speed + 0.4 × norm_accuracy

```

Version A restricts “latency” to the production-relevant calls only (initial prompter + routing, excluding the Judge, which never runs in a deployed assistant) and ranks Qwen3.5-2B first (0.980 s, score 0.960), narrowly ahead of gemma-4-E2B-it (1.450 s, score 0.959). Version B instead uses the full end-to-end latency, including the Judge call, and inverts that ranking: gemma-4-E2B-it comes first (0.734 s total, score 0.973) because its Judge call is the fastest and most uniform of any candidate (0.729 s init / 0.722 s route / 0.740 s judge – the three components within 20 ms of each other), ahead of Qwen3.5-2B (0.643 s total but a comparatively heavier 0.802 s judge call, score 0.960).

The two Ministral models and Phi-3.5-mini are excluded from the final recommendation on accuracy and latency grounds respectively (36–61% accuracy for Ministral; a 2.0–5.2 s latency for Phi-3.5-mini that both scoring versions penalize heavily), leaving six viable candidates. Among those, gemma-4-E2B-it is recommended as the balanced default: 95.8% accuracy, 0.734 s average end-to-end latency, the most uniform per-call latency profile of any candidate, and, at 2B parameters, the smallest realistic memory footprint among the models that clear 95% accuracy.

NOTE ON THE MEMORY-FOOTPRINT ARGUMENT

This last point is an engineering judgment call rather than a script output: neither `jetson_model_selector.py` nor its judge-aware counterpart computes an actual VRAM budget. The ~1.5 GB (2B-class) versus ~6.5 GB (9B-class) estimates used to argue that Qwen3.5-9B does not comfortably fit the target board’s shared memory budget are hand-derived from parameter count and Q4_K_M’s per-parameter footprint, not measured by the scripts themselves.

Qwen3.5-4B remains the recommended alternative whenever the deployment can accept roughly double the latency (1.16–1.72 s depending on scoring version) in exchange for the highest raw accuracy observed (96.3%).

Conclusion et perspectives

This internship set out to answer a scaling problem – how to validate an automotive voice assistant’s responses across many domains and languages without a human or a brittle keyword script in the loop – and delivered a working answer: a single local LLM, orchestrated as a two-node LangGraph state graph, that plays the Prompter, Router, and Judge roles well enough to benchmark nine candidate models end-to-end on a 216-case multilingual suite and turn the result into a concrete deployment recommendation (gemma-4-E2B-it, with Qwen3.5-4B as the higher-accuracy alternative). The engineering process behind that result was driven throughout by concrete, dated feedback – the three-metric latency split, the 216-case dataset target, the Gemma 4 family, the PASS/FAIL hallucination-vs-wrong-action distinction – each of which is now traceable to a specific line in the pipeline. A second, largely independent deliverable – an authorized security assessment of the target Jetson-class board – applied the same reconnaissance-to-verification discipline to the hardware side of the deployment and surfaced two real OpenSSH vulnerabilities requiring remediation before that board is exposed on a vehicle network.

The clearest lesson from the implementation was a gap between the design and what shipped: the original architecture called for a standalone Router node so that, per the two-tier routing idea in the original roadmap, a smaller and cheaper model could eventually be swapped in for that specific decision. What was actually built folds that decision into the Prompter’s own second call instead, which works – the benchmark’s turn-match rate is 100% for eight of nine models – but forecloses that specific future optimization until the Router logic is pulled back out into its own node. That, together with the roadmap items originally scoped for later milestones but not reached within this internship, defines the immediate next steps:

1. **Two-tier micro-model routing** – a small, fast model handling simple turn-taking decisions while the larger model is reserved for generation and judging, which requires first separating the Router back out as its own node.
2. **Asynchronous shadow judging** – decoupling the next voice command (needed immediately, to avoid a DUT microphone timeout) from the Judge’s slower semantic evaluation of the previous turn, which the latency breakdown in §3.5 shows is consistently the single most expensive call.
3. **Latency targets validated against real voice-activity-detection (VAD) timeouts** rather than the current, DUT-agnostic averages, once the pipeline is exercised against a live DUT through the existing `live/` graph rather than only the scripted offline harness.

4. **A telemetry/alignment dashboard** comparing the Judge's verdicts against a sample of human-graded transcripts, to give the QA and product teams a continuously-updated confidence figure rather than a one-off benchmark snapshot.
5. **Next-command speculative caching** – pre-generating the model's most likely follow-up commands while waiting on the DUT's transcript, to further shave the perceived latency of the Router loop.

Bibliography

- [1] LangChain AI. *LangGraph Documentation*. 2024. URL: <https://langchain-ai.github.io/langgraph/>.
- [2] Georgi Gerganov and Contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: <https://github.com/ggml-org/llama.cpp>.
- [3] Pydantic Contributors. *Pydantic: Data Validation Using Python Type Hints*. 2024. URL: <https://docs.pydantic.dev/>.
- [4] Qualys Threat Research Unit. *CVE-2024-6387: RegreSSHion – Remote Unauthenticated Code Execution in OpenSSH server*. 2024. URL: <https://nvd.nist.gov/vuln/detail/CVE-2024-6387>.
- [5] OpenSSH Project. *CVE-2025-26466: OpenSSH Uncontrolled Resource Consumption*. 2025. URL: <https://nvd.nist.gov/vuln/detail/CVE-2025-26466>.
- [6] Unsloth AI. *Qwen3.5 and gemma-4 GGUF model repositories*. Hugging Face. 2025. URL: <https://huggingface.co/unsloth>.
- [7] NVIDIA. *Jetson Platform Documentation*. 2025. URL: <https://developer.nvidia.com/embedded/jetson>.
- [8] OpenSSH Project. *OpenSSH*. 2025. URL: <https://www.openssh.com/>.

APPENDIX A

Security Assessment of the Target Edge Hardware

As a second, largely independent deliverable, an authorized defensive vulnerability assessment of the target Jetson-class board was carried out, following a standard reconnaissance → enumeration → vulnerability identification → proof-of-concept (PoC) verification methodology. An `nmap` scan of the in-scope host found five open ports: 22 (OpenSSH 9.6), 80 (nginx 1.24.0), 7777 (unidentified service), 50052 (unidentified service), and 50314 (Nagios NSCA, a monitoring-agent protocol). Manual enumeration of the port 80 web application found four endpoints – `/auth/login` (the login page itself), and `/logs`, `/update`, `/network`, each of which correctly 302-redirects an unauthenticated request back to `/auth/login` rather than exposing its content, indicating the application-layer authentication gate is at least present and functioning for those three management endpoints.

Version fingerprinting of the exposed OpenSSH 9.6 service matched it against two known CVEs, summarized in Table A.1. CVE-2024-6387 (“regreSSHion”) is a signal-handler race condition: if a client fails to authenticate within the login grace period, `sshd`’s asynchronous `SIGALRM` handler calls functions that are not `async-signal-safe`, which can corrupt memory in a way that ultimately allows unauthenticated, remote code execution with the privileges of the SSH daemon – rated Critical, CVSS 8.1. This finding was confirmed with a working proof-of-concept exploit run against the in-scope host, going beyond banner-based version matching alone. CVE-2025-26466 is an uncontrolled memory-consumption denial-of-service: the affected OpenSSH version allocates a reply in memory for every ping packet a client sends during key exchange, without freeing it until the exchange completes, letting a malicious client exhaust the server’s memory by simply never completing that handshake – rated High, CVSS 5.9. This second finding was identified through version fingerprinting against public advisories rather than independently exploited: no working PoC for CVE-2025-26466 was run against the host, so its presence should be read as a confirmed version-level exposure rather than a demonstrated live exploit.

Table A.1. Security assessment findings on the in-scope Jetson-class host.

Finding	Description	CVSS	Verification
Open ports	22 (OpenSSH 9.6), 80 (nginx 1.24.0), 7777, 50052, 50314 (Nagios NSCA)	—	nmap scan
Web endpoints	/auth/login (login page); /logs, /update, /network — each 302-redirects to /auth/login when unauthenticated	—	Manual enumeration
CVE-2024-6387 (regreSSHion)	Unauthenticated remote code execution via a signal-handler race condition in OpenSSH	8.1 Critical	PoC executed against the in-scope host
CVE-2025-26466	Uncontrolled memory consumption (denial of service) in OpenSSH	5.9 High	Version fingerprinting against public advisories; not independently exploited (no PoC run)

The recommended remediation is the same for both findings and is standard for an unauthenticated, network-facing service on outdated software: upgrade OpenSSH past the version in which both CVEs were fixed, and, in the interim or in addition, restrict access to the SSH port to a trusted management network rather than leaving it reachable from the vehicle's general network segment. The three unauthenticated management-plane ports discovered alongside SSH (7777, 50052, 50314) should be treated with the same caution even though no vulnerability was confirmed on them specifically in this assessment's scope: an unidentified open service is itself worth closing off or restricting to a management VLAN as a defense-in-depth measure, and the nginx application's endpoint-level authentication should be double-checked to ensure it cannot be bypassed by a client that skips the redirect and requests the underlying resource directly.